

LINQ Distinct Method in C# with Examples

In this article, I will discuss the **LINQ Distinct Method in C#** using Examples. Please read our previous article discussing the basics of [LINQ Set Operators](#). At the end of this article, you will understand the following pointers.

1. [What is LINQ Distinct Method in C#?](#)
2. [Example to Understand LINQ Distinct Method on Value Type](#)
3. [Example to Understand LINQ Distinct Method with String Values](#)
4. [LINQ Distinct Operation with Complex Data Type](#)
5. [Implementing IEqualityComparer Interface](#)
6. [Overriding Equals\(\) and GetHashCode\(\) Methods](#)
7. [Using Anonymous Type](#)
8. [Implementing IEquatable<T> Interface](#)
9. [When should we use the LINQ Distinct Method in C#?](#)

What is LINQ Distinct Method in C#?

The Distinct method in LINQ (Language Integrated Query) is a method used to remove duplicate elements from a collection. It's commonly used when working with arrays, lists, or any other type of IEnumerable in .NET. When you call Distinct on a collection, it returns a new collection that contains unique elements from the original collection based on the default equality comparer for the type of elements in the collection or a specified comparer. Two overloaded versions are available for the Distinct Method, as shown below.

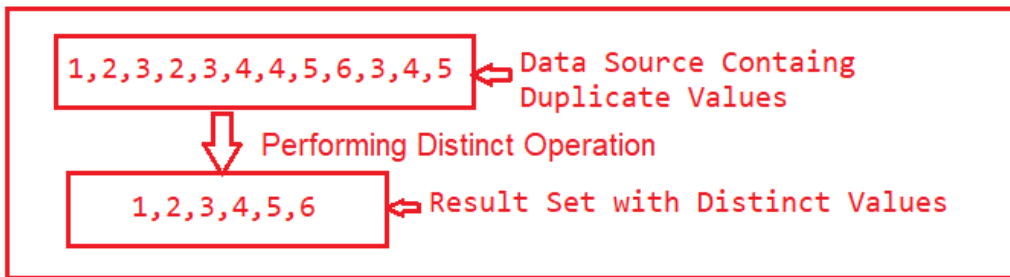
```
public static IEnumerable<TSource> Distinct<TSource>(this IEnumerable<TSource>
source);

public static IEnumerable<TSource> Distinct<TSource>(this IEnumerable<TSource>
source, IEqualityComparer<TSource> comparer);
```

The only difference between these two methods is that the second overloaded version takes an **IEqualityComparer** as an input parameter, which means the Distinct Method can also be used with Comparer.

Example to Understand LINQ Distinct Method on Value Type Using C#

Here, we have an integer collection that contains duplicate integer values. We are required to remove the duplicate values and return only the distinct ones, as shown below.



The following example shows how to get the distinct integer values from the data source using both Method and Mixed syntax using the LINQ Distinct Extension Method. No such operator called distinct is available in Query Syntax, so we need to use both Query and Method syntax to achieve the same.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> intCollection = new List<int>()
            {
                1,2,3,2,3,4,4,5,6,3,4,5
            };
            //Using Method Syntax
            var MS = intCollection.Distinct();
            //Using Query Syntax
            var QS = (from num in intCollection
                      select num).Distinct();
            foreach (var item in MS)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
1
2
3
4
5
6
```

How LINQ Distinct Method Works in C#?

When we call the Distinct Method on a collection, it internally uses a set data structure to keep track of elements that have already been encountered. It iterates over the elements of the collection, and for each element, it checks whether the set already contains an element equal to it.

To determine if an element is a duplicate, Distinct relies on the default equality comparer for the type of elements in the collection. This usually means using the Equals and GetHashCode methods of the elements.

Performance Considerations:

The performance of the Distinct method depends on the efficiency of the equality comparison. Since it uses a set for storing unique elements, the complexity is generally $O(n)$ for n elements in the collection, assuming a good distribution of hash codes for the elements.

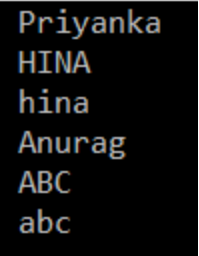
Example to Understand LINQ Distinct Method with String Values:

Let us see how we can use the LINQ Distinct Method with string values. In the below example, we have a string array of names, and we need to return the distinct names from that array collection. To do so, we are using the LINQ Distinct Method.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] namesArray = { "Priyanka", "HINA", "hina", "Anurag", "Anu",
            "ABC", "abc" };
            var distinctNames = namesArray.Distinct();
            foreach (var name in distinctNames)
            {
                Console.WriteLine(name);
            }

            Console.ReadKey();
        }
    }
}
```

When we execute the above program, it gives us the below output.



```
Priyanka
HINA
hina
Anurag
ABC
abc
```

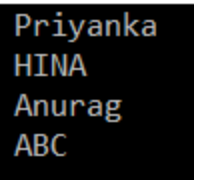
As you can see, the name Hina and Abc have appeared twice. This is because the default comparer, used by the LINQ Distinct method to filter the duplicate values, is case-sensitive. So, if you want to make the comparison case-insensitive, you need to use the other overloaded version of the Distinct Method, which takes **IEqualityComparer** as an argument. So here, we must pass a class that implements the **IEqualityComparer** interface.

So, let's modify the Program class as follows. Here, you can see we are passing **StringComparer** as an argument to the LINQ Distinct method and saying OrdinalIgnoreCase, which means, please ignore the case sensitive while checking the duplicity.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] namesArray = { "Priyanka", "HINA", "hina", "Anurag", "Ani",
            "ABC", "abc" };
            var distinctNames = namesArray.Distinct(StringComparer.OrdinalIgnoreCase);
            foreach (var name in distinctNames)
            {
                Console.WriteLine(name);
            }

            Console.ReadKey();
        }
    }
}
```

With the above changes in place, now run the application, and it should display the distinct names as shown in the below image.



```
Priyanka
HINA
Anurag
ABC
```

Now, if we go to the definition of the **StringComparer** class, you can see that this class implements the **IEqualityComparer** interface, as shown below. This is the reason why we can pass this class as a parameter to the Distinct Method.

```
public abstract class StringComparer : IComparer, IEqualityComparer, IComparer<string>,
IEqualityComparer<string>
```

LINQ Distinct Operation with Complex Data Type using C#:

The LINQ Distinct Method in C# will work differently with complex data types like Employee, Product, Student, etc. Let us understand this with an example. Create a class file named **Student.cs** and copy and paste the following code.

```
using System.Collections.Generic;
namespace LINQDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public static List<Student> GetStudents()
        {
            List<Student> students = new List<Student>()
            {
                new Student {ID = 101, Name = "Preety" },
                new Student {ID = 102, Name = "Sambit" },
                new Student {ID = 103, Name = "Hina"},
                new Student {ID = 104, Name = "Anurag"},
                new Student {ID = 102, Name = "Sambit"},
                new Student {ID = 103, Name = "Hina"},
                new Student {ID = 101, Name = "Preety" },
            };
            return students;
        }
    }
}
```

Here, we created the student class with the two properties, i.e., ID and Name. Along the same way, we have also created the GetStudents() method, which will return a hard-coded collection of students. So, basically, it returns the following student data.

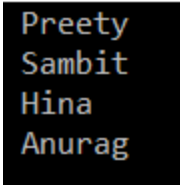
```
ID = 101, Name = "Preety"
ID = 102, Name = "Sambit"
ID = 103, Name = "Hina"
ID = 104, Name = "Anurag"
ID = 102, Name = "Sambit"
ID = 103, Name = "Hina"
ID = 101, Name = "Preety"
```

Example to Understand LINQ Distinct Method with Complex Type in C#:

Let us Understand the LINQ Distinct Method with Complex Type in C# with an example. Our requirement is to fetch all the distinct names from the student's collection. The following example shows how to use the LINQ Distinct Method to achieve the same using both Method and Query Syntax.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MS = Student.GetStudents()
                .Select(std => std.Name)
                .Distinct().ToList();
            //Using Query Syntax
            var QS = (from std in Student.GetStudents()
                select std.Name)
                .Distinct().ToList();
            foreach(var item in MS)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:



```
Preety
Sambit
Hina
Anurag
```

In our previous example, we tried retrieving the distinct student names, which worked as expected. We are required to select distinct students (ID and Name) from the collection. As you can see in our collection, three students are identical, and in our result set, they should appear only once. Let us modify the program class as follows to fetch the distinct students using the LINQ Distinct Method in C#.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    //Using Method Syntax
    var MS = Student.GetStudents()
        .Distinct().ToList();
    //Using Query Syntax
    var QS = (from std in Student.GetStudents()
        select std)
        .Distinct().ToList();
    foreach (var item in QS)
    {
        Console.WriteLine($"ID : {item.ID} , Name : {item.Name} ");
    }
    Console.ReadKey();
}
}
}

```

Now execute the query and see the output.

```

ID : 101 , Name : Preety
ID : 102 , Name : Sambit
ID : 103 , Name : Hina
ID : 104 , Name : Anurag
ID : 102 , Name : Sambit
ID : 103 , Name : Hina
ID : 101 , Name : Preety

```

As you can see, it will not select distinct students. Rather, it selects all the students. This is because the default comparer, used for comparison by the LINQ Distinct Method, only checked whether two object references are equal or not and not the individual property values of the complex object.

How to Solve the Above Problem?

We can solve the above problem in four different ways. They are as follows

1. We need to use the other overloaded version of the **Distinct()** method, which takes the **IEqualityComparer** interface as an argument. So, here, we need to create a class that implements the **IEqualityComparer** interface, and then we need to pass that comparer instance to the **Distinct()** method.
2. In the second approach, we need to override the **Equals()** and **GetHashCode()** methods within the **Student** class itself.
3. In the third approach, we must project the required properties into a **new anonymous type**, which overrides the **Equals()** and **GetHashCode()** methods.
4. By Implementing **IEquatable<T>** interface.

Approach 1: Implementing IEqualityComparer Interface

So, create a class file with the name **StudentComparer.cs**, implement the **IEqualityComparer** interface, and provide the implementation for Equals and GetHashCode Methods, as shown in the code below. Here, within the Equals Method, we are comparing the property values; if the property values are the same, we need to return true. Otherwise, we need to return false. Also, before accessing the values from the object, we need to ensure that the object itself is not null. Within the GetHashCode Method, we check the Student Object's hash value. And whenever we implement the Equals Method, we must also implement the GetHashCode.

```
using System.Collections.Generic;
namespace LINQDemo
{
    public class StudentComparer : IEqualityComparer<Student>
    {
        public bool Equals(Student x, Student y)
        {
            //First check if both object reference are equal then return true
            if(object.ReferenceEquals(x, y))
            {
                return true;
            }
            //If either one of the object reference is null, return false
            if (object.ReferenceEquals(x,null) || object.ReferenceEquals(y, null))
            {
                return false;
            }
            //Comparing all the properties one by one
            return x.ID == y.ID && x.Name == y.Name;
        }
        public int GetHashCode(Student obj)
        {
            //If obj is null then return 0
            if (obj == null)
            {
                return 0;
            }
            //Get the ID hash code value
            int IDHashCode = obj.ID.GetHashCode();
            //Get the string HashCode Value
            //Check for null reference exception
            int NameHashCode = obj.Name == null ? 0 : obj.Name.GetHashCode();
            return IDHashCode ^ NameHashCode;
        }
    }
}
```

Now, we need to create an instance of the **StudentComparer** class, and then we need to pass that instance to the Distinct method. So, modify the Main Method of the Program class as shown below.

```
using System;
using System.Linq;
namespace LINQDemo
```

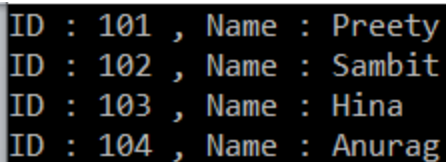
```

{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating an instance of StudentComparer
            StudentComparer studentComparer = new StudentComparer();

            //Using Method Syntax
            var MS = Student.GetStudents()
                .Distinct(studentComparer).ToList();
            //Using Query Syntax
            var QS = (from std in Student.GetStudents()
                select std)
                .Distinct(studentComparer).ToList();
            foreach (var item in QS)
            {
                Console.WriteLine($"ID : {item.ID} , Name : {item.Name} ");
            }
            Console.ReadKey();
        }
    }
}

```

With the above changes in place, now run the application, and it should display the distinct students as expected, as shown in the image below.



```

ID : 101 , Name : Preety
ID : 102 , Name : Sambit
ID : 103 , Name : Hina
ID : 104 , Name : Anurag

```

Approach 2: Overriding Equals() and GetHashCode() Methods within the Student Class

As we already know, any type in .NET is inherited from the Object class. That means the Student class is also inherited from the Object class. And we also know that the Object class provides virtual methods such as Equals() and GetHashCode(). We can override the **Equals()** and **GetHashCode()** methods of the Object class within the Student class. So, modify the Student class as shown below. We are overriding the Equals() and GetHashCode() methods here.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public static List<Student> GetStudents()

```

```

{
    List<Student> students = new List<Student>()
    {
        new Student {ID = 101, Name = "Preety" },
        new Student {ID = 102, Name = "Sambit" },
        new Student {ID = 103, Name = "Hina"},
        new Student {ID = 104, Name = "Anurag"},
        new Student {ID = 102, Name = "Sambit"},
        new Student {ID = 103, Name = "Hina"},
        new Student {ID = 101, Name = "Preety" },
    };
    return students;
}
public override bool Equals(object obj)
{
    //As the obj parameter type is object, so we need to
    //cast it to Student Type
    return this.ID == ((Student)obj).ID && this.Name == ((Student)obj)
}
public override int GetHashCode()
{
    //Get the ID hash code value
    int IDHashCode = this.ID.GetHashCode();
    //Get the string HashCode Value
    //Check for null refernece exception
    int NameHashCode = this.Name == null ? 0 : this.Name.GetHashCode();
    return IDHashCode ^ NameHashCode;
}
}
}

```

With the above changes in the Student class, now modify the Main method of the Program class as shown below. We don't need to do anything special with the Distinct Method.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MS = Student.GetStudents()
                .Distinct().ToList();
            //Using Query Syntax
            var QS = (from std in Student.GetStudents()
                select std)
                .Distinct().ToList();
            foreach (var item in MS)
            {

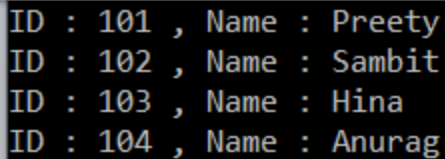
```

```

        Console.WriteLine($"ID : {item.ID} , Name : {item.Name} ");
    }
    Console.ReadKey();
}
}
}

```

Now execute the above program, and it will display the distinct records as expected, as shown in the below image.



```

ID : 101 , Name : Preety
ID : 102 , Name : Sambit
ID : 103 , Name : Hina
ID : 104 , Name : Anurag

```

Approach 3: Using Anonymous Type

In this approach, we need to project the properties of the Student class into a new anonymous type, and it will work as expected. The reason is the Anonymous Type already overrides the **Equals()** and **GetHashCode()** methods of the Object Class. So, modify the Main Method of the Program class as follows. Here, you can see using the Select Projection Operator and Select Method, we are projecting the output to an anonymous type.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MS = Student.GetStudents()
                .Select(std => new { std.ID, std.Name })
                .Distinct().ToList();
            //Using Query Syntax
            var QS = (from std in Student.GetStudents()
                select std)
                .Select(std => new { std.ID, std.Name })
                .Distinct().ToList();
            foreach (var item in MS)
            {
                Console.WriteLine($"ID : {item.ID} , Name : {item.Name} ");
            }
            Console.ReadKey();
        }
    }
}

```

In the above example, we project the ID and Name properties to IEnumerable<'a> means to anonymous type, which already overrides the Equals and GetHashCode method. Now run the application, and you will see the output as expected, as shown in the below image.

```
ID : 101 , Name : Preety
ID : 102 , Name : Sambit
ID : 103 , Name : Hina
ID : 104 , Name : Anurag
```

Approach 4: Implementing IEquatable<T> Interface in Student Class.

In this approach, we need to implement the **IEquatable<T>** Interface in Student Class and provide implementation for the Equals Method of the IEquatable<T> Interface. We also need to override the GetHashCode method of the Object class. So, modify the Student class as shown below.

```
using System.Collections.Generic;
using System;
namespace LINQDemo
{
    public class Student : IEquatable<Student>
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public static List<Student> GetStudents()
        {
            List<Student> students = new List<Student>()
            {
                new Student {ID = 101, Name = "Preety" },
                new Student {ID = 102, Name = "Sambit" },
                new Student {ID = 103, Name = "Hina"},
                new Student {ID = 104, Name = "Anurag"},
                new Student {ID = 102, Name = "Sambit"},
                new Student {ID = 103, Name = "Hina"},
                new Student {ID = 101, Name = "Preety" },
            };
            return students;
        }
        public bool Equals(Student other)
        {
            if (object.ReferenceEquals(other, null))
            {
                return false;
            }
            if (object.ReferenceEquals(this, other))
            {
                return true;
            }
            return this.ID.Equals(other.ID) && this.Name.Equals(other.Name);
        }
    }
}
```

```

        public override int GetHashCode()
        {
            int IDHashCode = this.ID.GetHashCode();
            int NameHashCode = this.Name == null ? 0 : this.Name.GetHashCode();
            return IDHashCode ^ NameHashCode;
        }
    }
}

```

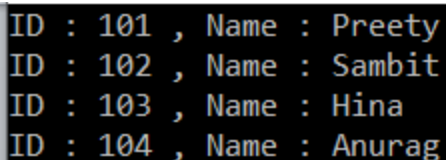
As you can see, here we have done two things. First, we implement the Equals method of the IEquatable interface and then override the GetHashCode method of the Object class. With the above changes in place, now modify the Main Method of the Program class as shown below.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MS = Student.GetStudents()
                .Distinct().ToList();
            //Using Query Syntax
            var QS = (from std in Student.GetStudents()
                select std)
                .Distinct().ToList();
            foreach (var item in MS)
            {
                Console.WriteLine($"ID : {item.ID} , Name : {item.Name} ");
            }
            Console.ReadKey();
        }
    }
}

```

Run the application, and you should see the output as expected, as shown in the below image.



```

ID : 101 , Name : Preety
ID : 102 , Name : Sambit
ID : 103 , Name : Hina
ID : 104 , Name : Anurag

```

Key Points to Remember While Working with LINQ Distinct Method:

- **Default Equality Comparer:** By default, Distinct uses the default equality comparer for the type of elements in the collection. For simple types like integers, strings, etc., this works as expected. For

complex types (like custom classes), you need to implement `IEquatable<T>` or provide a custom `IEqualityComparer<T>`.

- **Order Preservation:** The `Distinct` method preserves the order of the elements as they appear in the source collection. The first occurrence of each element is retained.
- **Deferred Execution:** Like many LINQ methods, `Distinct` uses deferred execution. The actual operation of removing duplicates is not performed until you iterate over the collection.
- **Null Values:** If the collection contains null values, `Distinct` treats them as equal, so only one null will be retained in the resulting collection.
- **Custom Equality Comparer:** For complex types or specific comparison needs, you can pass a custom equality comparer to the `Distinct` method.
- **Chaining with Other LINQ Methods:** `Distinct` can be used in combination with other LINQ methods like `Where`, `Select`, `OrderBy`, etc., making it a versatile tool for querying and manipulating collections.

When should we use the LINQ Distinct Method in C#?

The `Distinct` method is useful in scenarios where you have a list, array, or any other enumerable collection that may contain duplicate items, and you need to ensure that each item is unique. Here are some common use cases:

- **Removing Duplicates:** The most straightforward use of `Distinct` is to eliminate duplicate items from a collection. For example, if you have a list of integers with some repeated numbers and you want only unique numbers, `Distinct` can be used.
- **Data Processing:** In data processing tasks, especially when dealing with large datasets, duplicates might skew your results or lead to redundant processing. Using `Distinct` helps in getting a unique set of data for accurate processing.
- **Database Queries:** When querying a database using LINQ, `Distinct` can be used to ensure that the result set contains unique rows, especially when joins might produce duplicate rows.
- **Generating Reports:** When generating reports or summaries where duplicate values might lead to incorrect conclusions, `Distinct` helps in getting a distinct set of values for accurate reporting.
- **Set Operations:** In scenarios where you're performing set operations (like union, intersection, etc.) and need to ensure uniqueness, `Distinct` can be used after the set operation to ensure that the resulting set contains no duplicates.
- **Data Cleaning:** In data cleaning processes, `Distinct` is often used to remove duplicates as a part of making data consistent and reliable for further analysis.
- **Creating Unique Collections:** When you need to create a collection (like a list or array) of unique elements from an existing collection, `Distinct` is a convenient way to achieve this.

In the next article, I will discuss the [LINQ Except Method using C#](#) with Examples. I hope this article gives you a good understanding of the Concept of the LINQ Distinct Method in C# with Examples.